

- (1) $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_q, \gamma \xleftarrow{\mathbb{R}} \mathbb{Z}_q, u \leftarrow g^\alpha, w \leftarrow g^\gamma$
- (2) obtain (v, k) from $\mathbf{C}_{\text{kdf}}$

Additionally,

when $\mathcal{A}$ outputs $\hat{b}$:
 if $b = \hat{b}$ then output 1 else output 0

We leave it to the reader to verify (11.18).

Observe that in Game 2, the key k is only used to encrypt the challenge plaintext. As such, the adversary is essentially just playing the SS game with respect to $\mathcal{E}_s$ at this point. We leave it to the reader to describe an efficient SS adversary $\mathcal{B}_s$ that uses $\mathcal{A}$ as a subroutine, such that

$$|\Pr[W_2] - 1/2| = \text{SSadv}^*[\mathcal{B}_s, \mathcal{E}_s]. \quad (11.19)$$

Combining (11.16), (11.17), (11.18), and (11.19), yields (11.15), which completes the proof of the theorem. $\square$

11.6 A fun application: oblivious transfer based on Diffie-Hellman

Bob likes to be well informed. Every day he logs into his favorite newspaper site, reads the headlines, and downloads the articles that he wants to read. The newspaper charges Bob a few cents for every article that he reads, a price that Bob is happy to pay to support the newspaper. All articles have the same price.

One day Bob realizes that the newspaper can track what articles he is downloading and use this information to learn his political leanings, age group, and other personal information that Bob would rather keep private. He asks his friend Alice the following question:

The newspaper has articles $m_1, \dots, m_n \in \mathcal{M}$ and I am interested in reading article number $i \in \{1, \dots, n\}$. Can I download article m_i without the newspaper learning what article I downloaded?

Alice says that the newspaper can simply send Bob all the articles $m_1, \dots, m_n$ and Bob will read the ones he is interested in. The newspaper does not learn what articles Bob reads.

When Bob asked the newspaper to implement this scheme, they refused saying that this is a sure way for the newspaper to go out of business. Their readers will pay a few cents for a single article and get the entire newspaper. The newspaper would go along with the proposal if it could ensure that Bob only receives one article per request.

Oblivious transfer. Let's call the newspaper the *sender* and call Bob the *receiver*. Bob wants a solution to the following problem:

The sender has data $m_1, \dots, m_n \in \mathcal{M}$ and the receiver has an index $i \in \{1, \dots, n\}$. They want a protocol with the following property: when the protocol completes the receiver learns m_i and nothing else, while the sender learns nothing about i .

This problem is called 1-out-of- n **oblivious transfer**, or simply 1-out-of- n **OT**. It comes up in many settings beyond newspapers, such as privately buying digital content from an online store without the store learning what item is being purchased. More generally, OT is useful whenever there is a need to privately request a record in a proprietary database. It is also a central building block in many cryptographic protocols, as we will see in Chapter 23.

A warning. Defining security for OT protocols is quite subtle. Here we present two very simple OT protocols and discuss their security somewhat informally. We caution that both of these protocols require modification to ensure that they can be used securely when running several instances of the protocols concurrently, and when using them as subprotocols in arbitrary applications. Our goal here is to just get across some of the essential ideas, not to present protocols that should be deployed “as is” in arbitrary applications. We will discuss these security issues in Chapter 23.

11.6.1 A secure OT from ElGamal encryption

Our first 1-out-of- n OT protocol satisfies the basic OT security properties in the random oracle model, assuming only CDH. We let $\mathcal{M}$ denote the message space for the sender’s messages, so that $m_1, \dots, m_n \in \mathcal{M}$. The protocol uses the following ingredients:

- a cyclic group $\mathbb{G}$ of prime order q with generator $g \in \mathbb{G}$;
- a hash function $H : \mathbb{G}^2 \rightarrow \mathcal{K}$, which will be modeled as a random oracle;
- a semantically secure symmetric cipher (E_s, D_s) with key space $\mathcal{K}$ and message space $\mathcal{M}$.

We also assume that the sender and receiver communicate over a secure channel (i.e., one that utilizes authenticated encryption as discussed in Chapter 9).

With these ingredients in place, we are ready to describe an elegant OT protocol using ElGamal encryption in the group $\mathbb{G}$. We use a variant of the system $\mathcal{E}_{\text{EG}}$ from Section 11.5. The sender generates n ElGamal public keys $u_1, \dots, u_n \in \mathbb{G}$ and encrypts message m_j under public key u_j , for $j = 1, \dots, n$. The clever idea is a mechanism that ensures that the recipient knows the secret key for the public key u_i , but does not know the secret key for any the other key. This lets the recipient decrypt the i th ciphertext, obtaining m_i , but reveals nothing else. Let’s see how the protocol works and then discuss its security.

- *step 1:* the sender chooses $\beta \xleftarrow{\mathbb{R}} \mathbb{Z}_q$, computes $v \leftarrow g^\beta \in \mathbb{G}$, and sends v to the receiver.
- *step 2:* the receiver chooses $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_q$, computes $u \leftarrow g^\alpha v^{-i} \in \mathbb{G}$, and sends u to the sender.
- *step 3:* for $j = 1, \dots, n$ the sender computes

$$\begin{array}{ll}
u_j \leftarrow u \cdot v^j \in \mathbb{G} & // \text{ construct an ElGamal public key } u_j = g^\alpha v^{j-i} \\
w_j \leftarrow u_j^\beta & // \text{ construct an ElGamal ciphertext } (v, c_j) = (g^\beta, c_j) \\
k_j \leftarrow H(v, w_j) \in \mathcal{K} & // \text{ using randomness } \beta \text{ and public key } u_j, \\
c_j \xleftarrow{\mathbb{R}} E_s(k_j, m_j) & // \text{ that is the encryption of } m_j \in \mathcal{M}
\end{array}$$

and sends the vector of ciphertexts $C := (c_1, \dots, c_n)$ to the receiver. Note that all n ElGamal ciphertexts $(v, c_1), \dots, (v, c_n)$ are generated using the same encryption randomness β .

- *step 4*: the receiver, who has the secret key α for the ElGamal public-key $u_i = g^\alpha$, decrypts c_i as follows:

compute $w \leftarrow v^\alpha$, $k \leftarrow H(v, w)$
output $m \leftarrow D_s(k, c_i)$

Let's verify that this works correctly. During decryption in step 4, the key k is computed as $k \leftarrow H(v, w)$, where $w = v^\alpha = g^{\alpha\beta}$. In step 3 the key k_i is computed as $k_i \leftarrow H(v, w_i)$, where

$$w_i = u_i^\beta = (uv^i)^\beta = ((g^\alpha v^{-i})v^i)^\beta = g^{\alpha\beta}.$$

Hence, when the parties honestly follow the protocol, we have $w_i = w$ and $k_i = k$, and thus the receiver correctly obtains m_i .

Is the protocol secure? The only information that the sender learns about the receiver's input $i \in \{1, \dots, n\}$ is the group element $u = g^\alpha v^{-i}$ computed in step 2. Because α is not used in any other message sent to the sender, the quantity g^α is uniformly distributed over the group $\mathbb{G}$, and therefore perfectly hides v^{-i} . Hence, the sender learns nothing about i , as required.

To argue that the receiver learns at most one message, it suffices to argue that the receiver can query the random oracle representing H at no more than one point of the form (v, w_j) for $j = 1, \dots, n$. This will ensure that the receiver learns at most one decryption key, and hence at most one message. We briefly argue that if the receiver could query the random oracle at two such points, say (v, w_{j_1}) and (v, w_{j_2}) , then we could break the CDH for $\mathbb{G}$. In particular, we can transform such a receiver into an algorithm that takes as input $v = g^\beta$ and produces a tuple

$$(u, j_1, j_2, \quad w_{j_1} = (uv^{j_1})^\beta, \quad w_{j_2} = (uv^{j_2})^\beta) \tag{11.20}$$

for some $j_1 \neq j_2$. Dividing the two right most terms in (11.20) one by the other, and raising the result to the power of $1/(j_1 - j_2)$ gives v^β which is the same as $g^{(\beta^2)}$. Hence, we get an algorithm that takes as input g^β and computes $g^{(\beta^2)}$. We showed in Exercise 10.17 that this problem is equivalent to CDH in $\mathbb{G}$. We conclude that if CDH holds in $\mathbb{G}$ then the receiver can learn at most one message.

Remark 11.2. As we cautioned at the beginning of the section, the protocol above must be modified in order to satisfy stronger security properties. See [10] for a similar protocol that can be proven to satisfy such properties. In particular, the above informal security analysis did not take into account the possibility of attacks that can arise when multiple instances of the protocol run concurrently. Exercise 11.20 discusses such attacks, and shows how to prevent them by modifying the protocol in two ways. First, in deriving the secret keys using the hash function H , the hash function should also take as input the identities of the two parties (and possibly some type of “channel binding” or “session ID”, as in Section 21.8). Second, instead of a semantically secure symmetric cipher, we should use a cipher that provides one-time authenticated encryption. $\square$

11.6.2 Adaptive oblivious transfer

Going back to our newspaper scenario, the OT protocol presented above satisfies the security properties we want. However, when trying to deploy it, the newspaper hits a serious performance problem. For every article that Bob wants to download, the newspaper has to re-encrypt all the articles in the newspaper under new keys and send all these fresh ciphertexts to Bob. The

computation and communication costs per download would be prohibitive. In case of an online bookstore, the entire bookstore would need to be re-encrypted for every purchase.

It would be much better if the newspaper could encrypt all the articles once and post the encrypted articles on its web site for anyone to download. Bob, and other readers, will download the entire encrypted newspaper. Later, every time Bob pays for an article, the newspaper will quickly open the article for Bob without learning which article was opened. The computation and communication costs should be independent of the number of articles n . Bob will repeat this for as many articles as he wants to read.

An OT protocol that operates this way is called an **adaptive OT**. The word “adaptive” refers to Bob choosing the next article to open in a way that depends on the previously opened articles. Here we describe a simple protocol that satisfies the basic adaptive OT security requirements. That is, a receiver (even a malicious one) learns at most one message per run of the protocol, and the sender (even a malicious one) learns nothing about which messages were opened. In addition, the protocol should ensure that a malicious sender cannot make the receiver open the wrong message. This will provide consistency: whenever one or several receivers request the i th message, they always get the same message.

As we cautioned above, this protocol must be modified in order to satisfy stronger security properties. See [79] for a protocol that can be proven to satisfy such properties.

Before describing the adaptive OT we need a quick interlude to describe a useful, related concept called an *oblivious PRF*.

11.6.3 Oblivious PRFs

Let F be a secure PRF defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$. Suppose that one party, the *sender*, has the PRF key $k \in \mathcal{K}$. Another party, the *receiver*, has an input $x \in \mathcal{X}$. An **oblivious PRF protocol**, or simply an **OPRF protocol**, is a protocol that lets the receiver learn the output $y = F(k, x)$ without learning anything else about the value of the PRF at any other input; in addition, the sender learns nothing about the input x . Moreover, the OPRF protocol may be run many times, allowing the receiver to learn the value of $F(k, \cdot)$ for several inputs. The sender should learn nothing about any of these inputs, and the receiver should learn nothing about the value of $F(k, \cdot)$ at any other inputs. Actually, all we will really require here is that the receiver cannot learn anything about the value of $F(k, \cdot)$ at ℓ inputs unless it interacts with the sender ℓ or more times.

We can construct a simple OPRF protocol from the secure PRF F' presented in Exercise 11.3. That PRF makes use of a cyclic group $\mathbb{G}$ of prime order q with generator $g \in \mathbb{G}$ and is defined as

$$F'(k, x) := H' \left(x, H(x)^k \right) \quad \text{for } k \in \mathbb{Z}_q, \ x \in \mathcal{X}. \quad (11.21)$$

Here $H : \mathcal{X} \rightarrow \mathbb{G}$ and $H' : \mathcal{X} \times \mathbb{G} \rightarrow \mathcal{Y}$ are hash functions modeled as random oracles. The key space is $\mathcal{K} = \mathbb{Z}_q$. In Exercise 11.3, you are asked to prove that this PRF is secure assuming CDH holds for $\mathbb{G}$, and we model H and H' as random oracles.

Protocol OPRF1

We now describe an OPRF protocol for F' , which we call *Protocol OPRF1*. Suppose that the sender has a key $k \in \mathbb{Z}_q$. In each run of the protocol, the receiver has an input $x \in \mathcal{X}$, and the sender and receiver interact (over a secure channel) as follows:

- *receiver*: choose $\rho \xleftarrow{\mathbb{R}} \mathbb{Z}_q \setminus \{0\}$, compute $v \leftarrow H(x)^\rho \in \mathbb{G}$, and send v to the sender.
- *sender*: compute $w \leftarrow v^k \in \mathbb{G}$ and send w to the receiver.
- *receiver*: compute and output $y \leftarrow H'(x, w^{1/\rho}) \in \mathcal{Y}$.

First, observe that if sender and receiver are honest, then in the last step, the receiver computes

$$w^{1/\rho} = (v^k)^{1/\rho} = \left((H(x)^\rho)^k \right)^{1/\rho} = H(x)^k,$$

and so the receiver ends up with the correct output.

Second, observe that because of the way the receiver computes v , the value v leaks no information to the sender about the input x : the sender just sees a random group element, statistically independent of x .

Now suppose the receiver is honest but the sender is malicious. In this case, the sender still learns nothing about x ; however, the sender could send a wrong value for w , causing the receiver to end up with an incorrect value for the output $y = F'(k, x)$. Thus, Protocol OPRF1 is only partially secure if the sender is malicious: while the sender does not learn the receiver's input, the receiver may end up with the wrong output.

Now consider how much information a possibly malicious receiver can obtain by interacting with the sender. We want to argue that a receiver cannot learn anything about the value of $F'(k, \cdot)$ at ℓ inputs unless it interacts with the sender at least ℓ times. This follows directly from a reasonable assumption called the **one-more Diffie-Hellman** or **1MDH** assumption, if we model H and H' as random oracles.

Very briefly, we can describe the 1MDH assumption in terms of the following attack game. In this game, the challenger chooses $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_q$, computes $u \leftarrow g^\alpha \in \mathbb{G}$, and sends u to the adversary. Next, the adversary makes a sequence of queries to the challenger, each of which can be one of the following types:

Challenge query: the challenger chooses $v \xleftarrow{\mathbb{R}} \mathbb{G}$, and sends v to the adversary.

CDH solve query: the adversary submits $\hat{v} \in \mathbb{G}$ to the challenger, who computes and sends $\hat{w} \leftarrow \hat{v}^\alpha$ to the adversary.

At the end of the game, the adversary outputs a list of distinct pairs, each of the form (i, w) , where i is a positive integer bounded by the number of challenge queries, and $w \in \mathbb{G}$. We call such a pair (i, w) *correct* if $w = v^\alpha$ and v is the challenger's response to the i th challenge query. We say the adversary wins the game if the number of correct pairs exceeds the number of CDH solve queries. The 1MDH assumption says that every efficient adversary wins this game with at most negligible probability. In particular, an efficient adversary who sees a number of challenges raised to the power α , cannot produce any other challenge raised to the power α .

Clearly, the 1MDH assumption implies the CDH assumption. It is also not too hard to see that the 1MDH assumption implies that in the above OPRF construction, a receiver cannot learn anything about the value of $F'(k, \cdot)$ at ℓ inputs unless it interacts with the sender at least ℓ times. In particular, the receiver cannot query the random oracle representing H' at ℓ distinct points of the form $(x, H(x)^k)$ unless it interacts with the sender at least ℓ times. We leave this argument as an exercise to the reader.

We note that in certain groups $\mathbb{G}$, winning in the 1MDH game can be slightly easier than computing discrete log in $\mathbb{G}$. In these groups there is also an attack on the scheme OPRF1 that is slightly faster than computing discrete log in $\mathbb{G}$. The details are in Section 16.1.4.

Remark 11.3. When using the scheme OPRF1, it is important that the sender verify that v is in $\mathbb{G}$ before responding, otherwise the sender's response could inadvertently expose the secret key k (e.g., using an attack similar to the one in Exercises 12.3 or 15.1). $\square$

Protocol OPRF2

As we saw, Protocol OPRF1 is only partially secure in the presence of a malicious sender. We can easily modify the protocol to get one that provides a stronger security property in the presence of a malicious sender. In particular, this modified protocol, which we call *Protocol OPRF2*, guarantees that if the receiver's computed output y is wrong, then it is in fact a random value in $\mathcal{Y}$ that is completely independent of the sender's view. The protocol begins by having the sender publish the value $u = g^k$, where k is the sender's secret key. In each run of the protocol, the receiver has an input $x \in \mathcal{X}$, and the sender and receiver interact (over a secure channel) as follows:

- *receiver:* choose $\rho \xleftarrow{\mathbb{R}} \mathbb{Z}_q \setminus \{0\}$, $\tau \xleftarrow{\mathbb{R}} \mathbb{Z}_q$, compute $v \leftarrow H(x)^\rho \cdot g^\tau \in \mathbb{G}$, and send v to sender. Note that this v is computed differently than in OPRF1.
- *sender:* compute $w \leftarrow v^k \in \mathbb{G}$ and send w to the receiver.
- *receiver:* compute and output $y \leftarrow H'(x, (w/u^\tau)^{1/\rho}) \in \mathcal{Y}$.

Observe that if sender and receiver are honest, then in the last step, the receiver computes

$$(w/u^\tau)^{1/\rho} = \left(v^k/u^\tau\right)^{1/\rho} = \left((H(x)^\rho \cdot g^\tau)^k/g^{k\tau}\right)^{1/\rho} = H(x)^k,$$

and so the receiver ends up with the correct output $y = F'(k, x)$.

Next, observe that because of the way the receiver computes v , the value v leaks no information to the sender about the input x , and also leaks no information about the exponent ρ .

Now suppose the receiver is honest but the sender is malicious. If the sender responds in the second step with a wrong value for w , say $w = v^k \cdot d$, where $d \neq 1$, then the receiver computes

$$(w/u^\tau)^{1/\rho} = \left(v^k d/u^\tau\right)^{1/\rho} = H(x)^k \cdot d^{1/\rho}.$$

As we observed above, the sender knows nothing about ρ , and so from the sender's point of view, the value $d^{1/\rho} \in \mathbb{G}$ is just a random group element (uniformly distributed over $\mathbb{G}$). It then follows that the output value y computed by the receiver is also just a random element of $\mathcal{Y}$.

Finally, suppose the sender is honest but the receiver is malicious. Just as for Protocol OPRF1, under the 1MDH assumption, if we model H and H' as random oracles, then the receiver cannot learn anything about the value of $F'(k, \cdot)$ at ℓ inputs unless it interacts with the sender ℓ or more times.

11.6.4 A simple adaptive OT from an oblivious PRF

We will need the following ingredients:

- A PRF F defined over $(\mathcal{K}_{\text{prf}}, \{1, \dots, n\}, \mathcal{K})$.
- An OPRF protocol for a PRF F , like Protocol OPRF2 in Section 11.6.3, which is secure against a malicious sender, in the sense that the receiver will either get a correct output or a random output.

- A symmetric cipher (E_s, D_s) defined over $(\mathcal{K}, \mathcal{M}, \mathcal{C})$ that provides one-time authenticated encryption.

The protocol works as follows. Recall that the sender has messages $m_1, \dots, m_n \in \mathcal{M}$.

- *step 1*: the sender begins by doing:

$k \xleftarrow{R} \mathcal{K}_{\text{prf}}$
 for $i = 1, \dots, n$: $k_i \leftarrow F(k, i)$, $c_i \xleftarrow{R} E_s(k_i, m_i)$ // *encrypt all messages*
 send $C := (c_1, \dots, c_n)$ to the receiver // *send all ciphertexts to the receiver*

- *step 2*: when the receiver wants m_i , the receiver and sender establish a secure channel between them and use the OPRF protocol to compute $F(k, i)$:

the sender has k and the receiver has $i \in \{1, \dots, n\}$,
 when done, the receiver has $k_i = F(k, i)$

Finally, the receiver computes $m_i \leftarrow D_s(k_i, c_i)$.

That's it. If both parties honestly follow the protocol then the receiver obtains m_i , as required. Moreover, step 2 can be repeated as many times as needed. Every time the receiver gets to open another message of its choice from the sender's list. The amount of traffic generated in each invocation of step 2 is relatively small, and in particular, is independent of the size of the sender's list.

As for security, first suppose that the sender is honest but the receiver is malicious. In this case, from the security property of the OPRF protocol, we know that the number of messages that the receiver can decrypt is no more than the number of times the receiver runs step 2 of the protocol.

Second, suppose that the receiver is honest but the sender is malicious. In this case, from the security property of the OPRF protocol, we know that the sender learns nothing about which ciphertexts get decrypted by the receiver. Moreover, we know that if the sender deviates from the OPRF protocol, the receiver will obtain a random decryption key, and by the ciphertext integrity property of a symmetric cipher, the receiver will reject the ciphertext with overwhelming probability.

A final word of caution. To conclude this section we note that there are some inherent limitations to the security of oblivious transfer against corrupt parties. Consider again our newspaper example. Suppose that a corrupt newspaper wants to learn if Bob is reading articles in the finance section of the newspaper. The newspaper could choose its input articles $m_1, \dots, m_n$ to the OT protocol so that all the articles in the finance section are blank, while articles in other sections are correct newspaper articles. Now, when Bob wants to read an article in the finance section he will carry out an OT with the newspaper and end up with a blank article, despite having paid the newspaper for the article. Bob will undoubtedly complain that the oblivious transfer failed, at which point the newspaper learns that Bob requested an article in the finance section. This violates OT privacy. Of course, Bob could preserve his privacy by not complaining, but that is difficult to enforce in real life.

11.7 Notes

Citations to the literature to be added.

11.8 Exercises

11.1 (From weak PRF to PRF via hashing). This exercise gives a simple application of the random oracle model that will be useful later. Let $F'(k, x)$ be a PRF defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ and suppose that F' is weakly secure (as in Definition 4.3). Let $H : \mathcal{M} \rightarrow \mathcal{X}$ be a hash function, and define a new PRF over $(\mathcal{K}, \mathcal{M}, \mathcal{Y})$:

$$F(k, m) := F'(k, H(m)).$$

Show that F is a secure PRF (in the standard sense of a secure PRF) when H is modeled as a random oracle. In particular, you should show that for every adversary $\mathcal{A}$ attacking F as a PRF, there exists an adversary $\mathcal{B}$ attacking F' as a weak PRF, which is an elementary wrapper around $\mathcal{A}$, such that $\text{PRF}^{\text{roadv}}[\mathcal{A}, F] \leq \text{wPRFadv}[\mathcal{B}, F']$.

11.2 (Simple PRF from DDH). Let $\mathbb{G}$ be a cyclic group of prime order q generated by $g \in \mathbb{G}$. Let $H : \mathcal{M} \rightarrow \mathbb{G}$ be a hash function, which we shall model as a random oracle (see Section 8.10.2). Let F be the PRF defined over $(\mathbb{Z}_q, \mathcal{M}, \mathbb{G})$ as follows:

$$F(k, m) := H(m)^k \text{ for } k \in \mathbb{Z}_q, m \in \mathcal{M}.$$

Show that F is a secure PRF in the random oracle model for H under the DDH assumption for $\mathbb{G}$. In particular, you should show that for every adversary $\mathcal{A}$ attacking F as a PRF, there exists a DDH adversary $\mathcal{B}$, which is an elementary wrapper around $\mathcal{A}$, such that $\text{PRF}^{\text{roadv}}[\mathcal{A}, F] \leq \text{DDHadv}[\mathcal{B}, \mathbb{G}] + 1/q$.

Hint: First show that DDH implies that the PRF $F'(k, v) := v^k$ defined over $(\mathbb{Z}_q, \mathbb{G}, \mathbb{G})$ is a *weak* PRF (as in Definition 4.3). Use Exercise 10.11 for this. Then use Exercise 11.1 to deduce that F is a secure PRF.

11.3 (Simple PRF from CDH). Continuing with Exercise 11.2, let $H' : \mathcal{M} \times \mathbb{G} \rightarrow \mathbb{G}$ be a hash function, which we again model as a random oracle. Let F' be the PRF defined over $(\mathbb{Z}_q, \mathcal{M}, \mathbb{G})$ as follows:

$$F'(k, m) := H' \left(m, H(m)^k \right) \text{ for } k \in \mathbb{Z}_q, m \in \mathcal{M}.$$

Show that F' is a secure PRF assuming CDH for $\mathbb{G}$, and when H and H' are modeled as random oracles. Section 11.6.2 gives an interesting application for this PRF, exploiting the fact that it can be evaluated obliviously.

Hint: Use the result of Exercise 10.5.

11.4 (Broken variant of RSA). Consider the following broken version of the RSA public-key encryption scheme: key generation is as in $\mathcal{E}_{\text{RSA}}$, but to encrypt a message $m \in \mathbb{Z}_n$ with public key $pk = (n, e)$ do $E(pk, m) := m^e$ in $\mathbb{Z}_n$. Decryption is done using the RSA trapdoor.

Clearly this scheme is not semantically secure. Even worse, suppose one encrypts a *random* message $m \in \{0, 1, \dots, 2^{64}\}$ to obtain $c := m^e \bmod n$. Show that for 35% of plaintexts in $[0, 2^{64}]$, an adversary can recover the complete plaintext m from c using only 2^{35} eth powers in $\mathbb{Z}_n$.

Hint: Use the fact that about 35% of the integers m in $[0, 2^{64}]$ can be written as $m = m_1 \cdot m_2$ where $m_1, m_2 \in [0, 2^{34}]$.

11.5 (Multiplicative ElGamal). Let $\mathbb{G}$ be a cyclic group of prime order q generated by $g \in \mathbb{G}$. Consider a simple variant of the ElGamal encryption system $\mathcal{E}_{\text{MEG}} = (G, E, D)$ that is defined over $(\mathbb{G}, \mathbb{G}^2)$. The key generation algorithm G is the same as in $\mathcal{E}_{\text{EG}}$, but encryption and decryption work as follows:

- for a given public key $pk = u \in \mathbb{G}$ and message $m \in \mathbb{G}$:

$$E(pk, m) := \beta \xleftarrow{\mathbb{R}} \mathbb{Z}_q, \quad v \leftarrow g^\beta, \quad e \leftarrow u^\beta \cdot m, \quad \text{output } (v, e)$$

- for a given secret key $sk = \alpha \in \mathbb{Z}_q$ and a ciphertext $(v, e) \in \mathbb{G}^2$:

$$D(sk, (v, e)) := e/v^\alpha$$

- Show that $\mathcal{E}_{\text{MEG}}$ is semantically secure assuming the DDH assumption holds in $\mathbb{G}$. In particular, you should show that the advantage of any adversary $\mathcal{A}$ in breaking the semantic security of $\mathcal{E}_{\text{MEG}}$ is bounded by 2ϵ , where ϵ is the advantage of an adversary $\mathcal{B}$ (which is an elementary wrapper around $\mathcal{A}$) in the DDH attack game.
- Show that $\mathcal{E}_{\text{MEG}}$ is not semantically secure if the DDH assumption does not hold in $\mathbb{G}$.
- Show that $\mathcal{E}_{\text{MEG}}$ has the following property: given a public key pk , and two ciphertexts $c_1 \xleftarrow{\mathbb{R}} E(pk, m_1)$ and $c_2 \xleftarrow{\mathbb{R}} E(pk, m_2)$, it is possible to create a new ciphertext c which is an encryption of $m_1 \cdot m_2$. This property is called a **multiplicative homomorphism**.

11.6 (An attack on multiplicative ElGamal). Let p and q be large primes such that q divides $p - 1$. Let $\mathbb{G}$ be the order q subgroup of $\mathbb{Z}_p^*$ generated by $g \in \mathbb{G}$ and assume that the DDH assumption holds in $\mathbb{G}$. Suppose we instantiate the ElGamal system from Exercise 11.5 with the group $\mathbb{G}$. However, plaintext messages are chosen from the entire group $\mathbb{Z}_p^*$ so that the system is defined over $(\mathbb{Z}_p^*, \mathbb{G} \times \mathbb{Z}_p^*)$. Show that the resulting system is not semantically secure.

11.7 (Extending the message space). Suppose that we have a public-key encryption scheme $\mathcal{E} = (G, E, D)$ with message space $\mathcal{M}$. From this, we would like to build an encryption scheme with message space $\mathcal{M}^2$. To this end, consider the following encryption scheme $\mathcal{E}^2 = (G^2, E^2, D^2)$, where

$$\begin{aligned} G^2() &:= (pk_0, sk_0) \xleftarrow{\mathbb{R}} G(), \quad (pk_1, sk_1) \xleftarrow{\mathbb{R}} G(), \\ &\quad \text{output } pk := (pk_0, pk_1) \text{ and } sk := (sk_0, sk_1) \\ E^2(pk, (m_0, m_1)) &:= (E(pk_0, m_0), E(pk_1, m_1)) \\ D^2(sk, (c_0, c_1)) &:= (D(sk_0, c_0), D(sk_1, c_1)) \end{aligned}$$

Show that $\mathcal{E}^2$ is semantically secure, assuming $\mathcal{E}$ itself is semantically secure.

11.8 (Encrypting many messages with multiplicative ElGamal). Consider again the multiplicative ElGamal scheme in Exercise 11.5. To increase the message space from a single group element to several, say n , group elements, we could proceed as in the previous exercise. However, the following scheme, $\mathcal{E}_{\text{MMEG}} = (G, E, D)$ defined over $(\mathbb{G}^n, \mathbb{G}^{n+1})$, is more efficient.

- the key generation algorithm runs as follows:

$$\begin{aligned} G() &:= \alpha_i \xleftarrow{\mathbb{R}} \mathbb{Z}_q, \quad u_i \leftarrow g^{\alpha_i} \quad (i = 1, \dots, n) \\ &\quad pk \leftarrow (u_1, \dots, u_n), \quad sk \leftarrow (\alpha_1, \dots, \alpha_n) \\ &\quad \text{output } (pk, sk) \end{aligned}$$

- for a given public key $pk = (u_1, \dots, u_n) \in \mathbb{G}^n$ and message $m = (m_1, \dots, m_n) \in \mathbb{G}^n$:

$$\begin{aligned} E(pk, m) := & \quad \beta \xleftarrow{\mathbb{R}} \mathbb{Z}_q, \quad v \leftarrow g^\beta \\ & e_i \leftarrow u_i^\beta \cdot m_i \quad (i = 1, \dots, n) \\ & \text{output } (v, e_1, \dots, e_n) \end{aligned}$$

- for a given secret key $sk = (\alpha_1, \dots, \alpha_n) \in \mathbb{Z}_q^n$ and a ciphertext $c = (v, e_1, \dots, e_n) \in \mathbb{G}^{n+1}$:

$$D(sk, c) := (e_1/v^{\alpha_1}, \dots, e_n/v^{\alpha_n})$$

- (a) Show that $\mathcal{E}_{\text{MMEG}}$ is semantically secure assuming the DDH assumption holds in $\mathbb{G}$. In particular, you should show that the advantage of any adversary $\mathcal{A}$ in breaking the semantic security of $\mathcal{E}_{\text{MMEG}}$ is bounded by $2(\epsilon + 1/q)$, where ϵ is the advantage of an adversary $\mathcal{B}$, which is an elementary wrapper around $\mathcal{A}$, in the DDH attack game.

Hint: Use Exercise 10.11 with $n = 1$.

- (b) Theorem 11.1 shows that semantic security implies CPA security, but the concrete security bound degrades by a factor equal to the number Q of encryption queries. Show that for $\mathcal{E}_{\text{MMEG}}$, the security bound for CPA security only degrades by a factor of n , which is typically much smaller than Q . In particular, show that for every CPA adversary $\mathcal{A}$ there is a DDH adversary $\mathcal{B}$, which is an elementary wrapper around $\mathcal{A}$, such that $\text{CPAadv}[\mathcal{A}, \mathcal{E}_{\text{MMEG}}]$ is bounded by $2n \cdot (\epsilon + 1/q)$, where ϵ is the advantage of $\mathcal{B}$ in the DDH attack game.

Hint: Use Exercise 10.12.

11.9 (Modular hybrid construction). Both of the encryption schemes presented in this chapter, $\mathcal{E}_{\text{TDF}}$ in Section 11.4 and $\mathcal{E}_{\text{EG}}$ in Section 11.5, as well as many other schemes used in practice, have a “hybrid” structure that combines an asymmetric component and a symmetric component in a fairly natural and modular way. The symmetric part is, of course, the symmetric cipher $\mathcal{E}_s = (E_s, D_s)$, defined over $(\mathcal{K}, \mathcal{M}, \mathcal{C})$. The asymmetric part can be understood in abstract terms as what is called a **key encapsulation mechanism**, or **KEM**.

A KEM $\mathcal{E}_{\text{kem}}$ consists of a tuple of algorithms $(G, E_{\text{kem}}, D_{\text{kem}})$. Algorithm G is invoked as $(pk, sk) \xleftarrow{\mathbb{R}} G()$. Algorithm E_{kem} is invoked as $(k, c_{\text{kem}}) \xleftarrow{\mathbb{R}} E_{\text{kem}}(pk)$, where $k \in \mathcal{K}$ and $c_{\text{kem}} \in \mathcal{C}_{\text{kem}}$. Algorithm D_{kem} is invoked as $k \leftarrow D_{\text{kem}}(sk, c_{\text{kem}})$, where $k \in \mathcal{K} \cup \{\text{reject}\}$ and $c_{\text{kem}} \in \mathcal{C}_{\text{kem}}$. We say that $\mathcal{E}_{\text{kem}}$ is **defined over** $(\mathcal{K}, \mathcal{C}_{\text{kem}})$. We require that $\mathcal{E}_{\text{kem}}$ satisfies the following **correctness requirement**: for all possible outputs (pk, sk) of $G()$, and all possible outputs (k, c_{kem}) of $E_{\text{kem}}(pk)$, we have $D_{\text{kem}}(sk, c_{\text{kem}}) = k$.

We can define a notion of semantic security in terms of an attack game between a challenger and an adversary $\mathcal{A}$, as follows. In Experiment b , for $b = 0, 1$, the challenger computes

$$(pk, sk) \xleftarrow{\mathbb{R}} G(), \quad (k_0, c_{\text{kem}}) \xleftarrow{\mathbb{R}} E_{\text{kem}}(pk), \quad k_1 \xleftarrow{\mathbb{R}} \mathcal{K},$$

and sends (k_b, c_{kem}) to $\mathcal{A}$. Finally, $\mathcal{A}$ outputs $\hat{b} \in \{0, 1\}$. As usual, if W_b is the event that $\mathcal{A}$ outputs 1 in Experiment b , we define $\mathcal{A}$'s **advantage** with respect to $\mathcal{E}_{\text{kem}}$ as $\text{SSadv}[\mathcal{A}, \mathcal{E}_{\text{kem}}] := |\Pr[W_0] - \Pr[W_1]|$, and if this advantage is negligible for all efficient adversaries, we say that $\mathcal{E}_{\text{kem}}$ is **semantically secure**.

Now consider the **hybrid public-key encryption scheme** $\mathcal{E} = (G, E, D)$, constructed out of $\mathcal{E}_{\text{kem}}$ and $\mathcal{E}_s$, and defined over $(\mathcal{M}, \mathcal{C}_{\text{kem}} \times \mathcal{C})$. The key generation algorithm for $\mathcal{E}$ is the same as that of $\mathcal{E}_{\text{kem}}$. The encryption algorithm E works as follows:

$$E(pk, m) := \{ (k, c_{\text{kem}}) \xleftarrow{\mathcal{R}} E_{\text{kem}}(pk), c \xleftarrow{\mathcal{R}} E_s(k, m), \text{ output } (c_{\text{kem}}, c) \}.$$

The decryption algorithm D works as follows:

$$D(sk, (c_{\text{kem}}, c)) := \{ m \leftarrow \text{reject}, k \leftarrow D_{\text{kem}}(sk, c_{\text{kem}}), \text{ if } k \neq \text{reject then } m \leftarrow D_s(k, c), \\ \text{output } m \}.$$

- (a) Prove that $\mathcal{E}$ satisfies the correctness requirement for a public key encryption scheme, assuming $\mathcal{E}_{\text{kem}}$ and $\mathcal{E}_s$ satisfy their corresponding correctness requirements.
- (b) Prove that $\mathcal{E}$ is semantically secure, assuming that $\mathcal{E}_{\text{kem}}$ and $\mathcal{E}_s$ are semantically secure. You should prove a concrete security bound that says that for every adversary $\mathcal{A}$ attacking $\mathcal{E}$, there are adversaries $\mathcal{B}_{\text{kem}}$ and $\mathcal{B}_s$ (which are elementary wrappers around $\mathcal{A}$) such that

$$\text{SSadv}[\mathcal{A}, \mathcal{E}] \leq 2 \cdot \text{SSadv}[\mathcal{B}_{\text{kem}}, \mathcal{E}_{\text{kem}}] + \text{SSadv}[\mathcal{B}_s, \mathcal{E}_s].$$

- (c) Describe the KEM corresponding to $\mathcal{E}_{\text{TDF}}$ and prove that it is semantically secure (in the random oracle model, assuming $\mathcal{T}$ is one way).
- (d) Describe the KEM corresponding to $\mathcal{E}_{\text{EG}}$ and prove that it is semantically secure (in the random oracle model, under the CDH assumption for $\mathbb{G}$).
- (e) Let $\mathcal{E}_a = (G, E_a, D_a)$ be a public-key encryption scheme defined over $(\mathcal{K}, \mathcal{C}_a)$. Define the KEM $\mathcal{E}_{\text{kem}} = (G, E_{\text{kem}}, D_a)$, where

$$E_{\text{kem}}(pk) := \{ k \xleftarrow{\mathcal{R}} \mathcal{K}, c_{\text{kem}} \xleftarrow{\mathcal{R}} E_a(pk, k), \text{ output } (k, c_{\text{kem}}) \}.$$

Show that $\mathcal{E}_{\text{kem}}$ is semantically secure, assuming that $\mathcal{E}_a$ is semantically secure.

Discussion: Part (e) shows that one can always build a KEM from a public-key encryption scheme by just using the encryption scheme to encrypt a symmetric key; however, parts (c) and (d) show that there are more direct and efficient ways to do this.

11.10 (Multi-key CPA security). Generalize the definition of CPA security for a public-key encryption scheme to the multi-key setting. In this attack game, the adversary gets to obtain encryptions of many messages under many public keys. Show that semantic security implies multi-key CPA security. You should show that security degrades linearly in $Q_k Q_e$, where Q_k is a bound on the number of keys, and Q_e is a bound on the number of encryption queries per key. That is, the advantage of any adversary $\mathcal{A}$ in breaking the multi-key CPA security of a scheme is at most $Q_k Q_e \cdot \epsilon$, where ϵ is the advantage of an adversary $\mathcal{B}$ (which is an elementary wrapper around $\mathcal{A}$) that breaks the scheme's semantic security.

11.11 (A tight reduction for multiplicative ElGamal). We proved in Exercise 11.10 that semantic security for a public-key encryption scheme implies multi-key CPA security; however, the security degrades significantly as the number of keys and encryptions increases. Consider the

multiplicative ElGamal encryption scheme $\mathcal{E}_{\text{MEG}}$ from Exercise 11.5. You are to show a *tight* reduction from multi-key CPA security for $\mathcal{E}_{\text{MEG}}$ to the DDH assumption, which does not degrade at all as the number of keys and encryptions increases. In particular, you should show that the advantage of any adversary $\mathcal{A}$ in breaking the multi-key CPA security of $\mathcal{E}_{\text{MEG}}$ is bounded by $2(\epsilon + 1/q)$, where ϵ is the advantage of an adversary $\mathcal{B}$ (which is an elementary wrapper around $\mathcal{A}$) in the DDH attack game.

Note: You should assume that in the multi-key CPA game, the same group $\mathbb{G}$ and generator $g \in \mathbb{G}$ is used throughout.

Hint: Use Exercise 10.11.

11.12 (An easy discrete log group). Let n be a large integer and consider the following subset of $\mathbb{Z}_{n^2}^*$:

$$\mathbb{G}_n := \{ [an + 1]_{n^2} \in \mathbb{Z}_{n^2}^* : a \in \{0, \dots, n-1\} \}$$

- Show that $\mathbb{G}_n$ is a multiplicative subgroup of $\mathbb{Z}_{n^2}^*$ of order n .
- Which elements of $\mathbb{G}_n$ are generators?
- Choose an arbitrary generator $g \in \mathbb{G}_n$ and show that the discrete log problem in $\mathbb{G}_n$ is easy.

11.13 (Paillier encryption). Let us construct another public-key encryption scheme (G, E, D) that makes use of RSA composites:

- The key generation algorithm is parameterized by a fixed value ℓ and runs as follows:

$G(\ell) :=$ generate two distinct random ℓ -bit primes p and q ,
 $n \leftarrow pq, \quad d \leftarrow (p-1)(q-1)/2$
 $pk \leftarrow n, \quad sk \leftarrow d$
output (pk, sk)

- for a given public key $pk = n$ and message $m \in \{0, \dots, n-1\}$, set $g := [n+1]_{n^2} \in \mathbb{Z}_{n^2}^*$. The encryption algorithm runs as follows:

$$E(pk, m) := \quad h \xleftarrow{\mathbb{R}} \mathbb{Z}_{n^2}^*, \quad c \xleftarrow{\mathbb{R}} g^m h^n \in \mathbb{Z}_{n^2}^*, \quad \text{output } c.$$

- Explain how the decryption algorithm $D(sk, c)$ works.

Hint: Using the notation of Exercise 11.12, observe that c^d falls in the subgroup $\mathbb{G}_n$ which has an easy discrete log.

- Show that this public-key encryption scheme is semantically secure under the following assumption:

let n be a product of two random ℓ -bit primes,
let u be uniform in $\mathbb{Z}_{n^2}^*$,
let v be uniform in the subgroup $(\mathbb{Z}_{n^2})^n := \{h^n : h \in \mathbb{Z}_{n^2}^*\},$

then the distribution (n, u) is computationally indistinguishable from the distribution (n, v) .

Discussion: This encryption system, called **Paillier encryption**, has a useful property called an additive homomorphism: for ciphertexts $c_0 \xleftarrow{\mathbb{R}} E(pk, m_0)$ and $c_1 \xleftarrow{\mathbb{R}} E(pk, m_1)$, the product $c \leftarrow c_0 \cdot c_1$ is an encryption of $m_0 + m_1 \bmod n$.

11.14 (Hash Diffie-Hellman). Let $\mathbb{G}$ be a cyclic group of prime order q generated by $g \in \mathbb{G}$. Let $H : \mathbb{G} \rightarrow \mathcal{K}$ be a hash function. We say that the **Hash Diffie-Hellman** (HDH) assumption holds for $(\mathbb{G}, H)$ if the distribution $(g^\alpha, g^\beta, H(g^\beta, g^\alpha))$ is computationally indistinguishable from the distribution (g^α, g^β, k) where $\alpha, \beta \xleftarrow{\mathbb{R}} \mathbb{Z}_q$ and $k \xleftarrow{\mathbb{R}} \mathcal{K}$.

- (a) Show that if H is modeled as a random oracle and the CDH assumption holds for $\mathbb{G}$, then the HDH assumption holds for $(\mathbb{G}, H)$.
- (b) Show that if H is a secure KDF and the DDH assumption holds for $\mathbb{G}$, then the HDH assumption holds for $(\mathbb{G}, H)$.
- (c) Prove that the ElGamal public-key encryption scheme $\mathcal{E}_{\text{EG}}$ is semantically secure if the HDH assumption holds for $(\mathbb{G}, H)$.

11.15 (Anonymous public-key encryption). Suppose t people publish their public-keys $pk_1, \dots, pk_t$. Alice sends an encrypted message to one of them, say pk_5 , but she wants to ensure that no one (other than user 5) can tell which of the t users is the intended recipient. You may assume that every user, other than user 5, who tries to decrypt Alice's message with their secret key, obtains fail.

- (a) Define a security model that captures this requirement. The adversary should be given t public keys $pk_1, \dots, pk_t$ and it then selects the message m that Alice sends. Upon receiving a challenge ciphertext, the adversary should learn nothing about which of the t public keys is the intended recipient. A system that has this property is said to be **an anonymous public-key encryption scheme**.
- (b) Show that the ElGamal public-key encryption system $\mathcal{E}_{\text{EG}}$ is anonymous.
- (c) Show that the RSA public-key encryption system $\mathcal{E}_{\text{RSA}}$ is not anonymous. Assume that all t public keys are generated using the same RSA parameters ℓ and e .

11.16 (Proxy re-encryption). Bob works for the Acme corporation and publishes a public-key pk_{bob} so that all incoming emails to Bob are encrypted under pk_{bob} . When Bob goes on vacation he instructs the company's mail server to forward all his incoming encrypted email to Alice. Alice's public key is pk_{alice} . The mail server needs a way to translate an email encrypted under public-key pk_{bob} into an email encrypted under public-key pk_{alice} . This would be easy if the mail server had sk_{bob} , but then the mail server can read all of Bob's incoming email.

Consider the variation $\mathcal{E}'_{\text{EG}}$ of $\mathcal{E}_{\text{EG}}$, in which we only hash w instead of (v, w) (see Remark 11.1). Suppose that pk_{bob} and pk_{alice} are public keys for $\mathcal{E}'_{\text{EG}}$. Then the mail server can do the translation from pk_{bob} to pk_{alice} while learning nothing about the email contents.

- (a) Suppose $pk_{\text{alice}} = g^\alpha$ and $pk_{\text{bob}} = g^{\alpha'}$. Show that giving $\tau := \alpha'/\alpha$ to the mail server lets it translate an email encrypted under pk_{bob} into an email encrypted under pk_{alice} , and vice-versa.
- (b) Assume that $\mathcal{E}'_{\text{EG}}$ is semantically secure. Show that the adversary cannot break semantic security for Alice, even if it is given Bob's public key $g^{\alpha'}$ along with the translation key τ .

11.17 (Online-offline oblivious transfer). In Section 11.6 we looked at protocols for oblivious transfer where one party, a sender, has messages $m_0, \dots, m_{n-1} \in \mathcal{M} = \{0, 1\}^\ell$ and another party, a

receiver, has an index $i \in \mathbb{Z}_n$. At the end of the protocol the receiver has m_i and neither party learns anything else about the other party's data. Suppose that the sender and receiver had previously established a random OT instance: the sender has random $x_0, \dots, x_{n-1} \xleftarrow{\mathbb{R}} \mathcal{M}$ and the recipient has (j, x_j) for some random $j \xleftarrow{\mathbb{R}} \mathbb{Z}_n$. This random data is called an **OT correlation**. Let's show that the parties can quickly solve the OT problem using a random OT correlation:

Sender has: $(m_1, \dots, m_{n-1}), (x_1, \dots, x_{n-1})$, Receiver has: (i, j, x_j) .
Receiver wants m_i .

- *receiver*: compute $\Delta \leftarrow (j - i) \in \mathbb{Z}_n$ and send Δ to the sender.
 - *sender*: for all $u = 0, 1, \dots, n - 1$ compute $c_u \leftarrow m_u \oplus x_{(u+\Delta)}$ and send $C := (c_0, \dots, c_{n-1})$ to the receiver. Here each index $(u + \Delta)$ is an element in $\mathbb{Z}_n$.
 - *receiver*: output $c_i \oplus x_j$.
- (a) Show that when both parties honestly follow the protocol, the receiver learns the required value m_i .
- (b) Prove that a (possibly malicious) sender learns nothing about the receiver's index i . Similarly, a (possibly malicious) receiver cannot learn anything about any messages besides m_i , where i is defined as $i := j - \Delta \in \mathbb{Z}_n$.
- (c) A protocol between the sender and the receiver that constructs a random OT correlation, but where the recipient can choose the $j \in \mathbb{Z}_n$ that it wants, is called a **Random OT** or **ROT protocol**. ROT is discussed in more detail in Section 23.7. Adapt the protocol in this exercise to show that an ROT protocol implies an OT protocol.

11.18 (All-but-one oblivious transfer). All-but-one OT refers to the following problem: a sender has messages $m_1, \dots, m_n \in \mathcal{M}$ and a receiver has an index $i \in \{1, \dots, n\}$. At the end of the protocol the receiver should have all of $m_1, \dots, m_n$ *except* for message m_i . Neither party should learn anything else about the other party's inputs. Here is a simple solution using a CPA secure cipher (E, D) defined over $(\mathcal{K}, \mathcal{M}, \mathcal{C})$:

- *sender*: choose a random k in $\mathcal{K}$ and compute $c_j \xleftarrow{\mathbb{R}} E(k, m_j)$ for $j = 1, \dots, n$.
 - *both*: the sender and receiver engage in n independent 1-out-of-2 oblivious transfers. In OT instance number $j = 1, \dots, n$, the sender's input data is the pair (c_j, k) , and the receiver's input data is a bit b_j in $\{0, 1\}$.
- (a) Explain why the receiver can learn at most $n - 1$ of the sender's messages, and why the sender learns nothing about i .
- (b) Generalize the protocol to allow the receiver to learn at most $n - 2$ of the sender's messages, where the sender learns nothing about which ones.

Hint: try using n independent 1-out-of-3 oblivious transfers.

Discussion: The all-but-one OT protocol described in this problem make use of n executions of 1-out-of-2 oblivious transfer. It is possible to implement all-but-one OT using only $\log_2 n$ executions of 1-out-of-2 oblivious transfer [142, Sec. 3].

11.19 (Expanding oblivious transfer). Suppose we are given a 1-out-of-2 oblivious transfer protocol Π . We wish to construct a 1-out-of- n oblivious transfer protocol, for $n > 2$, using only $\lceil \log_2 n \rceil$ parallel executions of the 1-out-of-2 OT protocol Π . For simplicity assume $n = 2^t$, for some $t > 1$. We will need a hash function $H : \mathcal{K}^t \rightarrow \mathcal{K}'$, where $\mathcal{K} := \{0, 1\}^w$, and a cipher (E, D) defined over $(\mathcal{K}', \mathcal{M}, \mathcal{C})$. The 1-out-of- n OT protocol Π' works as follows:

Sender has: $m_0, \dots, m_{n-1} \in \mathcal{M}$, Receiver has: $i \in \{0, \dots, n-1\}$.

Receiver wants m_i .

- *Setup:* The receiver computes the binary representation of i , namely $b_0, \dots, b_{t-1} \in \{0, 1\}$ so that $i = \sum_{j=0}^{t-1} 2^j b_j$. The sender chooses random $k_j[0], k_j[1] \xleftarrow{\mathcal{R}} \mathcal{K}$ for $j = 0, \dots, t-1$.
- *OT phase:* The sender and receiver engage in $t = \log_2 n$ parallel instances of the 1-out-of-2 OT protocol Π , where in instance number j , the receiver's input is $b_j \in \{0, 1\}$, and the sender's input is $(k_j[0], k_j[1]) \in \mathcal{K}^2$. At the end of the t protocols, the receiver has $k_0[b_0], \dots, k_{t-1}[b_{t-1}] \in \mathcal{K}$.
- *Sender:* For $\ell = 0, \dots, n-1$, the sender does:
 - let $d_0, \dots, d_{t-1} \in \{0, 1\}$ be the binary representation of ℓ ,
 - compute $k_\ell \leftarrow H(k_0[d_0], \dots, k_{t-1}[d_{t-1}]) \in \mathcal{K}'$,
 - set $c_\ell \xleftarrow{\mathcal{R}} E(k_\ell, m_\ell)$.

Send $\mathbf{c} = (c_0, \dots, c_{n-1})$ to the receiver.

- *Receiver:* set $\hat{k}_i \leftarrow H(k_0[b_0], \dots, k_{t-1}[b_{t-1}]) \in \mathcal{K}'$ and outputs $\hat{m} \leftarrow D(\hat{k}_i, c_i)$.

(a) Show that protocol Π' is correct so that $\hat{m} = m_i$.

(b) Let's prove a minimal security property for Π' : the receiver learns nothing about m_j for $j \neq i$. More precisely, consider an efficient adversary that outputs i , and two tuples $\mathbf{m} = (m_0, \dots, m_{n-1})$ and $\mathbf{m}' = (m'_0, \dots, m'_{n-1})$, with $m_i = m'_i$. The adversary then plays the role of receiver with input i . We say that Π' is secure if the adversary cannot distinguish the message $\mathbf{c}$ from the sender when the sender takes $\mathbf{m}$ as input versus when the sender takes $\mathbf{m}'$ as input. Show that when H is modeled as a random oracle, (E, D) is semantically secure, and the 1-out-of-2 OT protocol Π is secure, then the 1-out-of- n OT protocol Π' is secure.

Discussion: This protocol runs many instances of 1-out-of-2 OT. In Section 23.7 we will show how to efficiently construct many instances of 1-out-of-2 OT, as needed here, using a technique called OT extension.

11.20 (An attack on oblivious transfer). The first OT protocol presented in Section 11.6 is subject to a kind of “man in the middle” attack, when multiple instances of the protocol may run concurrently. However, the protocol is easily repaired to prevent this. Suppose we run *two* OT instances of the protocol: in one instance the adversary plays the role of receiver and interacts with an honest sender; in the other instance the adversary plays the role of sender and interacts with an honest receiver. The adversary relays all messages unchanged between the honest sender and honest receiver. However, when the honest receiver sends $u := g^\alpha v^{-i}$, the adversary sends $\hat{u} := u \cdot v$ to the honest sender.

- (a) Show that when the two OT protocols complete, the receiver incorrectly obtains the message m_{i-1} instead of m_i . You may assume $i > 1$.

- (b) Now suppose we modify the protocol along the lines discussed on Remark 11.2. Explain why the attack does not work any more.

Discussion: One of the reasons we use secure channels in this protocol is to prevent the adversary from modifying protocol messages. However, even with secure channels, the protocol as stated is subject to a “man in the middle attack” as illustrated in this exercise. Here, the honest sender and honest receiver are talking to the adversary in two separate protocol instances, and we would ideally like these two protocol instances to run relatively independently of each other, without any “strange interactions” between them, such as that illustrated in this exercise. This type of subtle attack, which arises when we run protocol instances concurrently, is one reason why we need better security definitions for interactive protocols, which we will discuss in Section 23.5.

11.21 (Group key exchange). Suppose n parties $A_1, \dots, A_n$ want to setup a group secret key that they can use for encrypted group messaging. They have at their disposal a public bulletin board that they can all post messages to and whose contents are public. No peer-to-peer communication is allowed. Your goal is to design a group key exchange that is secure against passive eavesdropping.

- (a) Use a public-key encryption scheme (G, E, D) to design a protocol that runs in two rounds. One party, say A_1 , reads and writes n values to and from the bulletin board. All other parties read and write only one value to the bulletin board.
- (b) Define a security model for a group key exchange protocol that ensures security against a passive eavesdropper. Prove security of your protocol from part (a) assuming (G, E, D) is semantically secure.
- (c) Design a fairer protocol where the work is evenly distributed. Specifically, design a protocol based on two-party Diffie-Hellman that runs in at most k rounds, assuming $n = 2^k$. Let $\mathbb{G}$ be a group of prime order q with generator $g \in \mathbb{G}$ and let $H : \mathbb{G} \rightarrow \mathbb{Z}_q$ be a hash function. In each round every party posts at most one group element in $\mathbb{G}$ to the bulletin board and reads at most one group element from the bulletin board. At the end of the protocol, after k rounds, all parties obtain the same secret key and there are at most $2n$ group elements on the bulletin board. You may assume that the parties know each other’s identities and can order themselves lexicographically by identity.

Hint: think of an n -leaf binary tree where each leaf corresponds to one party.

- (d) Prove security of your protocol from part (c) against a passive eavesdropper assuming CDH holds in $\mathbb{G}$ and H is modeled as a random oracle.
- (e) Suppose one of the n parties has a malfunctioning random number generator — whenever that party needs to sample a random value, its random number generator always returns “5”. Show that this will sink the entire protocol from parts (a) and (c), even if all the other participants have well functioning random number generators. Specifically, show that an eavesdropper will learn the group secret key.
- (f) Is it possible to design a group key exchange protocol that is secure even if at most one of the participants suffers from the problem in part (e)?